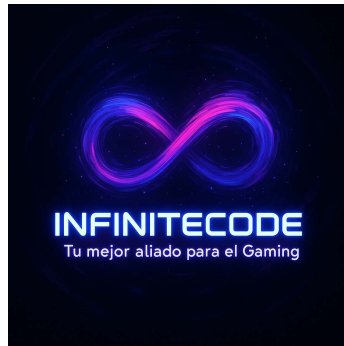




TÍTULO DEL PROYECTO: INFINITECODE (Pagina web)

FECHA: 1/11/2025((Presentacion))

VERSIÓN: 2.5 ((Actual))

ÍNDICE DE CONTENIDO

1. Introducción
2. Objetivos
3. Metodología de desarrollo
4. Herramientas y programas utilizados
5. Estructura del proyecto
6. Descripción funcional
7. Diseño visual

Introducción

En INFINITECODE nos comprometemos a revolucionar el mercado gaming, proporcionando una plataforma digital confiable que conecta apasionados gamers con los mejores productos y servicios del sector.

Objetivos

Nuestro objetivo es atender las problemáticas dadas en la industria actual, tales como la escasez de estas tiendas en el sector, el compromiso con el cliente y la accesibilidad para cualquier persona. Por ende se busca promover una comunidad sana, actualizada respecto a las novedades de la industria y demás.

Metodología de desarrollo

El desarrollo de esta página pasó por 3 etapas esenciales:

1. Contextualización o Planeación: Se analizaron y discutieron distintas propuestas basadas en aplicativos web populares como Amazon o Corseir.
2. Proceso de desarrollo: Se comenzó replicando los conocimientos anteriormente adquiridos, desde la presentación de productos hasta el carrito de compras, se pensó en todo momento en la comodidad del usuario, por ende el filtrado de los productos, sus categorías y sus descripciones fueron algo que se fue desarrollando poco a poco.
3. Presentación: Esta es la etapa actual donde se evalúa el trabajo ya realizado, se analiza los pros y contras para su respectivo mejoramiento y se tiene una concepción clara demostrable de "Qué es la página".

Herramientas y programas utilizados

Para la pagina web se uso el editor de codigo de Microsoft (**Visual studio code**) que facilitaba el manejo de los archivos y el codigo interno del proyecto gracias a sus multiples extensiones que mejoraban la experiencia de desarrollo.

Se uso multiples navegadores para las pruebas realizadas a lo larga de las versiones de la pagina, actualmente aun no se tiene un repositorio oficial, pues cuando esta este lista se podran subir todas las versiones realizadas hasta la fecha en **Git-hub**.

Estructura del proyecto

Tras abrir la carpeta Code Infinite Store se mostraran los siguientes archivos que conforman toda la estructura de la pagina web:

Nombre	Fecha de modifica...	Tipo	Tamaño
 Imagenes	1/11/2025 7:07 p. m.	Carpeta de archivos	
 Carrito	1/11/2025 7:13 p. m.	Chrome HTML Do...	31 KB
 Carritologic	14/06/2025 8:33 p....	Archivo JavaScript	8 KB
 chackeo	7/05/2025 1:25 p. m.	Archivo JavaScript	2 KB
 Filtrado	20/05/2025 8:41 p....	Archivo JavaScript	2 KB
 Gameplay	3/06/2016 11:50 a. ...	Archivo de fuente ...	22 KB
 INFINITECODE	1/11/2025 7:11 p. m.	Chrome HTML Do...	41 KB
 infinitecodeconfig	1/11/2025 6:03 p. m.	Archivo de origen ...	51 KB
 Nosotros	1/11/2025 7:09 p. m.	Chrome HTML Do...	60 KB
 Plantilla Formato IEEE 830 Completo.docx	1/11/2025 6:25 p. m.	Documento Adob...	1,633 KB
 Politicas_de_Privacidad_GamingCenter_v2	14/10/2025 7:56 p....	Documento Adob...	5 KB
 Términos y Condiciones (1)	14/10/2025 7:43 p....	Documento Adob...	102 KB
 Untitled-1	28/04/2025 5:12 p....	Chrome HTML Do...	1 KB
 Visualizador_Productos	1/11/2025 7:12 p. m.	Chrome HTML Do...	38 KB
 Visualizadorconfig	15/06/2025 11:34 a...	Archivo JavaScript	207 KB
 Vogue	13/07/2018 4:36 p....	Archivo de fuente ...	26 KB

Descripción detallada de archivos

CODE INFINITE STORE

Este documento recopila una descripción técnica y funcional de los archivos HTML, CSS y JavaScript presentes en el proyecto. Está pensado para que un desarrollador nuevo entienda la función de cada archivo, los patrones usados y las propiedades más relevantes.

Índice

1. HTML principales
 1. INFINITECODE.html
 2. Nosotros.html
 3. Carrito.html
 4. Visualizador_Productos.html
 5. Untitled-1.html
2. JavaScript
 1. Filtrado.JS
 2. Carritologic.js
 3. Visualizadorconfig.js
 4. chackeo / auth scripts
3. CSS principal: infinitecodeconfig.css
4. Assets y notas generales

HTML

INFINITECODE.html

Propósito: Página principal / landing. Muestra banner, productos, audio de fondo, acceso a login y enlaces globales.

Estructura clave:

- `<nav class="top-bar">` — barra superior con login, botones.
- `<nav class="navbar">` — navbar principal con logo, enlaces, dropdown/mega-menu.
- Banners: `.banner`, lista de productos: `.productos`.
- Se incluye reproductor de audio (`#bgMusic`) y controlador para reproducción automática/volumen.
- Incluye lateral de redes sociales: `.Redes`.

Patrones JS/CSS usados: IntersectionObserver para animaciones, modales controlados con clases (ej. `.activo`), background con video (clase `.video-background`), y uso extensivo de flex/grid para layout.

Nosotros.html

Propósito: Página de presentación del proyecto: fundadores, misión, visión, objetivos, diagramas, e índice interno (barra de separación con enlaces a anclas).

Estructura clave:

- Video de fondo: `.video-background`.
- Secciones tipo slide: cada una con `.slide` y encabezados `<h1>`.
- Índice interno: `<nav class="navbar-separator">` con enlaces `<a href="#mision">` y separadores `<span>/</span>`.

Javascript importante: script agregado para scroll suave desde los enlaces en `.navbar-separator`. Calcula el offset de las barras fijas (`.top-bar` y `.navbar`) y desplaza con `window.scrollTo({behavior: 'smooth'})`.

Notas: Los encabezados principales tienen ids (por ejemplo `id="mision"`) para que las anclas funcionen correctamente; la compensación del header se maneja en JS o con `scroll-margin-top` en CSS.

Carrito.html

Propósito: Interfaz del carrito de compras, resumen y finalización de compra.

Estructura clave:

- Contenedor de lista de carrito (por ejemplo `#carrito-lista`), total (`#carrito-total`) y botón de compra (`#comprar`).
- Scripts para mostrar historial de compras en modal, formularios de login/registro y edición.

Interacciones: Añadir/eliminar items, persistencia en `localStorage`, y guardado de historial por usuario (`historialCompras_`).

Visualizador_Productos.html

Propósito: Página de detalles del producto; recibe datos desde el listado (mediante `localStorage`) y muestra `#product-title`, `#product-description`, `#product-price`, `#product-image`.

Flujo: desde el listado de productos se llama a `viewProduct(...)` (en JS) que escribe el producto en `localStorage` y redirige a esta página; al cargar se lee y se rellena el DOM.

Untitled-1.html

Archivo de prueba / placeholder con HTML mínimo. No es parte crítica del sistema.

JavaScript

Filtrado.JS

Propósito: Conjunto de utilidades reusables (contador de carrito, menú, historial).

Funciones clave:

- `toggleMenu()` — muestra/oculta menú mediante `style.display`.
- Renderiza historial de compras desde `localStorage` en `#historial-container`.
- Delegación de eventos para eliminar registros y recarga.

APIs y patrones: `DOMContentLoaded`, `querySelector`, `createElement`, `dataset`, `localStorage`, `location.reload()`.

Carritologic.js

Propósito: Lógica completa del carrito: añadir, renderizar, eliminar, finalizar compra y persistencia.

Variables importantes: `let cart = JSON.parse(localStorage.getItem("cart")) || [];`

Funciones:

- `updateCartCount()` — suma cantidades y actualiza contador visual (`#carrito-count`).
- `renderCart()` — construye la lista de items en DOM (uso de `innerHTML` y `template literals`).
- `addToCart(producto)` — guarda en `localStorage` y actualiza visual.
- `finalizarCompra()` — valida `usuarioActivo`, guarda historial y limpia carrito.

Patrones: Delegación de eventos (captura de clicks en botones dinámicos), `Array.prototype.reduce`, persistencia con `localStorage`.

Visualizadorconfig.js

Propósito: Generación de catálogo de productos (array embebido) y funciones para ver/cargar producto en la vista detalle.

Funciones:

- `viewProduct(title,description,price,image)` — guarda campos en `localStorage` y redirige a `Visualizador_Productos.html`.
- `loadProductDetails()` — lee `localStorage` y rellena elementos del DOM con los datos del producto.

Datos: El array `productos` contiene docenas de objetos con `nombre`, `descripcion`, `precio`, `imagen`, `categoria`. Para escalar, conviene mover esta lista a un JSON o a un backend.

chackeo / auth snippets

Pequeñas utilidades para autenticación demo (ejemplo: usuario "admin" / "1234"). Guardan bandera en `localStorage` (`isAuthenticated`) y redirigen si no autenticado.

Nota de seguridad: Es solo demo; para producción usar backend y tokens seguros (JWT/OAuth).

CSS

infinitecodeconfig.css

Propósito: Hoja de estilos global principal que define layout, tipografías, paleta y componentes de UI.

Secciones y patrones principales:

- `@font-face` — define tipografías locales (Vogue, Gameplay).
- Componentes visuales: `.Logo`, `.Logotext`, `.NewText`, `.card`, `.banner`, `.productos`, `.Redes`.
- Navbar y navegación: `.top-bar`, `.navbar`, `.navbar-links`, `.navbar-separator`.
- Animaciones: `@keyframes` `fadeIn`, uso de `transform`, `opacity` y `transition`.
- Responsive: varias media queries para `max-width`: 768px y 1024px.

Propiedades CSS más usadas: `display: flex/grid, gap, box-shadow, border-radius,`
`background: linear-gradient(), position: absolute/fixed, z-index, text-shadow.`

Assets

Carpeta: Imágenes/

Contiene logos, iconos, imágenes de producto y videos (referenciados desde HTML y CSS).
Las rutas relativas se usan ampliamente (por ejemplo `./Imagenes/Logo.ico`).

Patrones comunes y recomendaciones

- Uso consistente de `localStorage` para persistencia de usuario, carrito, historial y transferencia de datos entre páginas.
- `IntersectionObserver` para animaciones de entrada (clase `visible` añadida al intersectar).
- Modales y formularios controlados por toggles de clase (`.activo`) o estilos inline (`style.display`).

Nota rápida: Se espera próximamente migrar el código a React.

Archivos Varios

PDF

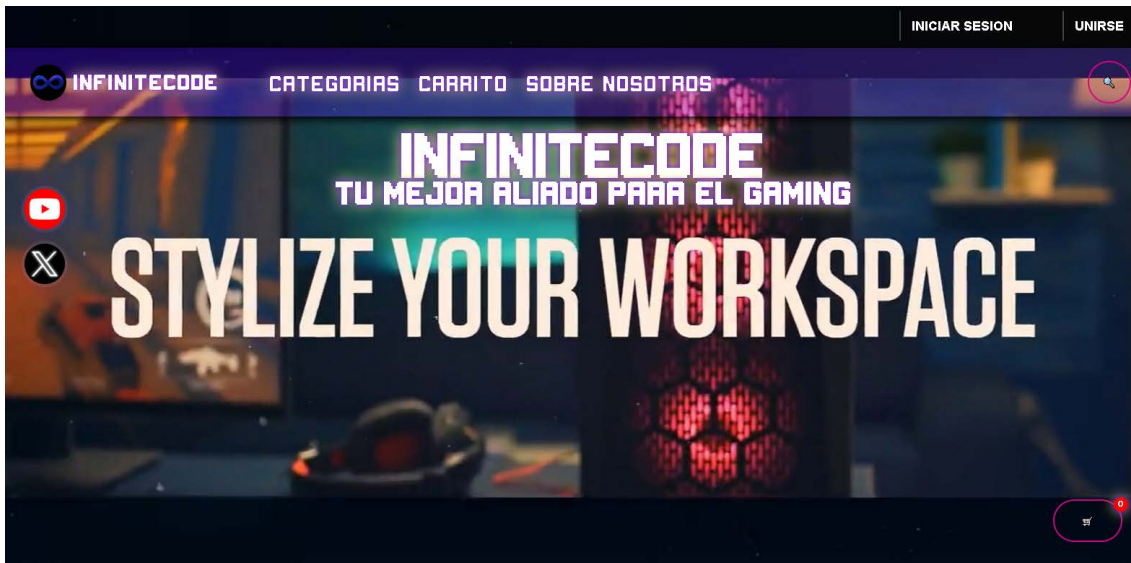
Formato IEE, Política de privacidad, terminos y condiciones.

Fuentes

Gameplay y Vogue.

Diseño

Inicio de sesion



Navbar

Redes

Banners

